

Master Judge Question Bank

IBM Bob Multi-Agent AI Software Team

Detailed, judge-safe answers for learning the full project and responding with confidence.

Practical coverage: 140+ likely questions, follow-ups, trade-offs, limitations, and presentation traps.

How to use this book

No document can literally predict every sentence a judge may say. This is an exhaustive practical question bank: it covers the questions most likely to arise from the slides, source code, architecture, evidence files, and real-world follow-ups. Learn the idea behind each answer instead of memorizing every word. If a question is outside scope, say what is implemented, what is not, and how you would approach it next.

Professional answer pattern: (1) state the direct answer, (2) point to the evidence or design reason, and (3) name the limitation or next step when relevant. Never exaggerate automation, security, or production readiness.

Contents

Section	Topic	Coverage
1	Pitch, problem, and value proposition	10 questions
2	Why multi-agent instead of one agent	10 questions
3	IBM Bob and custom modes	12 questions
4	Agent roles and permission boundaries	12 questions
5	Workflow controller and auditability	12 questions
6	FastAPI application and API design	12 questions
7	Data validation and database code	12 questions
8	Security, review, and code quality	12 questions
9	Testing, debugging, and CI	12 questions
10	Scope, productionization, and scaling	12 questions
11	Comparison, ethics, and credibility	12 questions
12	Demo execution, traps, and advanced follow-ups	15 questions

1. Pitch, problem, and value proposition

1.1 Q: What is your project in one sentence?

Answer: It is a governed multi-agent software-development workflow built with IBM Bob custom modes. Six specialized roles plan, implement, review, test, debug, and document a small FastAPI application while leaving evidence for the next stage.

1.2 Q: What problem are you solving?

Answer: AI can generate code quickly, but one general AI agent can also review and test its own work. That creates blind spots in security, requirements, and verification. Our workflow adds independent roles, restricted access, and explicit evidence checks.

1.3 Q: Why is this an important problem?

Answer: As AI-assisted development expands, organizations need more than a model saying 'this is complete.' They need reproducible validation, accountability, and a way to prevent one broad agent from silently changing every part of a project.

1.4 Q: What is the key innovation?

Answer: The key innovation is not the to-do API itself. It is the controlled process around it: specialized IBM Bob modes, file-level write boundaries, independent verification, human-approved hand-offs, and an audit trail.

1.5 Q: Why did you choose a to-do API?

Answer: A to-do API is familiar and deliberately small. It gives us real code, validation, persistence, error handling, and tests, but leaves the judges enough time to inspect the multi-agent workflow rather than learn a complex product domain.

1.6 Q: Who are the target users?

Answer: The immediate target is students, hackathon teams, and small software teams using AI coding assistants. The same governance pattern can later support enterprise teams that need stricter controls, traceability, and approval policies.

1.7 Q: What is the business value?

Answer: It can reduce rework and defect escape by catching problems before release. It also makes AI-assisted changes easier to review because each role has a visible output and a narrow responsibility.

1.8 Q: How is this different from simply using ChatGPT or any coding assistant?

Answer: A general chat assistant can help with all tasks, but it normally has no enforced separation between author, reviewer, tester, and approver. Our project uses IBM Bob custom modes to make these roles explicit and restrict what each mode can edit.

1.9 Q: What evidence shows that the workflow adds value?

Answer: Review Agent found a dynamic-SQL anti-pattern, scope drift, and a validation gap. A deliberately introduced DELETE status-code bug was caught by tests and precisely diagnosed by Debug Agent. These are stored in project reports, not merely claimed in slides.

1.10 Q: What should judges remember after the presentation?

Answer: We did not build only a to-do API. We built a controlled IBM Bob development team where every role has one job, one permission boundary, and one piece of evidence for the next role.

2. Why multi-agent instead of one agent

2.1 Q: Why not use one general AI agent?

Answer: One agent can write the code, decide it is secure, write only the tests it expects to pass, and claim success. Specialization introduces independent checks, similar to how real software teams separate implementation, review, quality assurance, and approval.

2.2 Q: Does more agents always mean better results?

Answer: No. More agents add coordination cost. A tiny, low-risk task may be faster with one agent. This approach is valuable when correctness, security, traceability, and independent verification matter more than raw speed.

2.3 Q: Why exactly six agents?

Answer: The six roles map to a complete but lightweight software lifecycle: clarify requirements, build, review, test, diagnose failures, and communicate verified results. Each role has a distinct artifact and decision point.

2.4 Q: How do agents communicate?

Answer: They communicate through explicit project artifacts such as `design_brief.md`, `review_report.md`, `test_report.md`, and `debug_report.md`. This is simpler and more auditable than relying on hidden conversational memory.

2.5 Q: Why not let the Coding Agent test its own code?

Answer: Developer tests are useful, but independent testing catches a different class of mistakes. In this project, Testing Agent cannot edit `app/main.py`, so it cannot silently change the product to turn a failure into a pass.

2.6 Q: Why not let Review Agent fix a problem immediately?

Answer: Separating review from repair preserves accountability. Review Agent reports the issue; Coding Agent makes the change; Review and Testing then verify the repair. That avoids a reviewer changing the evidence it is meant to assess.

2.7 Q: Why is Debug Agent separate from Coding Agent?

Answer: A test error says what failed, not always why. Debug Agent must determine whether the cause is an application bug, a test bug, or a specification mismatch before Coding Agent makes a focused repair.

2.8 Q: Can agents run in parallel?

Answer: Some work could be parallelized in a larger system, such as documentation research and security scanning. This proof of concept uses a sequence because later stages depend on evidence from earlier stages and because explicit hand-offs are part of the design.

2.9 Q: What prevents role overlap?

Answer: Role definitions and custom instructions state what each mode should do, while `fileRegex` scopes provide technical limits on which files each role may edit. The strongest boundary is not behavioral instruction; it is the permission configuration.

2.10 Q: What is the biggest trade-off of the multi-agent approach?

Answer: The trade-off is coordination overhead. We accept it because the project aims to demonstrate safer, more reviewable AI-assisted development rather than the fastest possible generation of a small API.

3. IBM Bob and custom modes

3.1 Q: What IBM Bob feature did you use?

Answer: We use IBM Bob custom modes configured in `.bob/custom_modes.yaml`. Each mode defines a role, when to use it, detailed instructions, allowed capability groups, and a `fileRegex`-based edit scope.

3.2 Q: What is a custom mode?

Answer: A custom mode is a persistent, configured agent role in IBM Bob. It has a purpose and tool permissions, so it is more structured than repeatedly typing a role prompt into a general chat.

3.3 Q: Where can you prove the modes exist?

Answer: The actual configuration is in `.bob/custom_modes.yaml`. The agent-config markdown files are human-readable descriptions, but the YAML configuration is the file that defines what Bob runs.

3.4 Q: What are `roleDefinition` and `whenToUse`?

Answer: `roleDefinition` describes who the agent is and the one job it owns. `whenToUse` tells the team at what point in the workflow that mode should be selected. Together they keep the roles understandable and repeatable.

3.5 Q: What are `customInstructions`?

Answer: They are the mechanical rules for each mode: which artifacts it must produce, required sections, and actions it must not take. For example, Debug Agent must write a root-cause report but not modify application code.

3.6 Q: What are groups in the YAML file?

Answer: `groups` defines which capability categories a mode may use, such as read, edit, execute, skill, or subtask. The edit group can be narrowed further by a file path regular expression.

3.7 Q: What is `fileRegex`?

Answer: `fileRegex` is a regular-expression pattern that matches the files a mode may edit. For example, `^app/.*` allows files under `app/`, while `^review_report\.md$` allows exactly one report file.

3.8 Q: How is Bob used in the actual workflow?

Answer: A team member manually selects the appropriate custom mode in IBM Bob, gives it the required input artifacts, and lets it produce its restricted output. The mode switch is deliberate and human-controlled.

3.9 Q: Does IBM Bob automatically orchestrate all six agents?

Answer: No, and we do not claim that it does. Bob provides the specialized modes and permission boundaries. Our controller validates hand-off evidence and records a human approval, but a person still selects the next mode.

3.10 Q: Why is manual selection not a weakness?

Answer: It preserves an explicit accountability point. For a proof of concept, manual mode selection is also more honest than claiming unsupported automatic orchestration. Future work can integrate approved gates with enterprise workflows.

3.11 Q: What did you learn about IBM Bob while building this?

Answer: We learned that custom-mode configuration must enumerate every artifact an agent needs. For example, Testing Agent initially could not write its root-level test report until its `fileRegex` was expanded to allow it.

3.12 Q: Could the same Bob configuration be reused?

Answer: Yes. The pattern is reusable: retain the roles and permission model, then change the domain-specific design brief, source folder, testing tools, and evidence gates for another project.

4. Agent roles and permission boundaries

4.1 Q: What does Research Agent do?

Answer: It reads the scope or feature request and writes `design_brief.md`. The brief includes the data model, endpoints, dependencies, edge cases, and out-of-scope items, but Research Agent does not write application code.

4.2 Q: What does Coding Agent do?

Answer: It reads the approved design brief and implements only what it specifies inside `app/`. When Debug Agent reports a failure, Coding Agent makes only the focused repair rather than unrelated refactoring.

4.3 Q: What does Review Agent do?

Answer: It checks implementation against the design and scope for security issues, scope violations, inconsistent error handling, and missing edge cases. It writes a verdict to `review_report.md` but does not edit source code.

4.4 Q: What does Testing Agent do?

Answer: It independently writes and runs API tests, then records results in `test_report.md`. It can write test files and the report, but not application source.

4.5 Q: What does Debug Agent do?

Answer: It runs only after a FAIL result. It identifies the actual cause, classifies it as `APP_BUG`, `TEST_BUG`, or `SPEC_MISMATCH`, and writes a recommendation with a confidence level.

4.6 Q: What does Pitch/Docs Agent do?

Answer: It runs only after a passing test report. It turns verified artifacts into `app/README.md` and `docs/demo-script.md` without changing code or retroactively changing evidence.

4.7 Q: Why can Coding Agent write `app/` but Testing Agent cannot?

Answer: Coding Agent owns implementation, while Testing Agent owns independent verification. This separation is the central control that prevents an AI role from both causing and concealing a defect.

4.8 Q: Can an agent edit files outside its scope by mistake?

Answer: The intent is that IBM Bob enforces the configured edit scope at the tool level. The project does not rely solely on the agent choosing to follow a sentence in its prompt.

4.9 Q: Why can Pitch/Docs Agent edit `app/README.md` even though docs should not edit code?

Answer: `README.md` is documentation located inside the `app` folder. Its `fileRegex` allows only that specific documentation path and `docs/demo-script.md`, not application source such as `app/main.py`.

4.10 Q: How would you add a Security Agent?

Answer: Add a new custom mode with a narrow role: inspect dependencies, code, and configuration; write a security report; and restrict its edits to that report. Then add a controller gate requiring the report before release.

4.11 Q: How would you add a DevOps Agent?

Answer: Give it a dedicated infrastructure folder and outputs such as deployment manifests or a deployment checklist. Keep it unable to change application code unless that is explicitly approved by the workflow.

4.12 Q: What is least privilege in this project?

Answer: Least privilege means granting only the minimum access needed to complete a role. Research can write one brief, Review can write one report, Testing can write tests, and only Coding can modify application source.

5. Workflow controller and auditability

5.1 Q: What is workflow/handoff_controller.py?

Answer: It is a small Python command-line tool that wraps the human-driven IBM Bob workflow with evidence checks, permitted transitions, approval records, and a JSONL audit trail.

5.2 Q: What does the controller automate?

Answer: It checks whether prior artifacts exist and contain the required approval or test result, permits only documented transitions, requires a named approver, writes an audit record, and prints the next Bob mode plus required inputs.

5.3 Q: What does the controller not automate?

Answer: It does not select a Bob mode, execute Bob on the team's behalf, edit application code, or bypass Bob permissions. A human remains responsible for approving and initiating the next agent.

5.4 Q: What is a gate?

Answer: A Gate is one allowed source-to-target transition plus the evidence check required to pass it. For example, review to testing uses the `_review_approved` check and points to Testing Agent as the next mode.

5.5 Q: Which transitions are allowed?

Answer: research to coding, coding to review, review to testing, testing to docs, testing to debug, and debug to coding. There is no direct jump from research to docs or from coding to docs.

5.6 Q: What is required for Review to Testing?

Answer: `review_report.md` must contain the exact approved verdict used by the controller check. If it does not, the controller blocks the hand-off rather than allowing testing to appear as a valid next step.

5.7 Q: What is required for Testing to Docs?

Answer: `test_report.md` must record PASS and zero failures. This makes documentation a post-verification activity rather than a claim written before the product has passed tests.

5.8 Q: What is required for Testing to Debug?

Answer: The test report must record FAIL. Debug is a failure-specific role, so it should not be used on a passing build merely to create extra activity.

5.9 Q: What is JSONL and why use it?

Answer: JSON Lines stores one JSON object per line. It is simple to append, easy for a human to read, easy for a script to parse, and suitable for a lightweight audit log.

5.10 Q: Is the audit log tamper-proof?

Answer: Not in this proof of concept. It is append-only by normal program behavior but remains a local file. Production hardening would include protected branches, signed commits, identity-backed approvals, and immutable centralized storage.

5.11 Q: What does the status command do?

Answer: It evaluates each stage's current evidence and writes `docs/workflow/status.md` as a snapshot. The JSONL hand-off log is the audit trail; the status file is a current readable summary.

5.12 Q: How would you improve the controller?

Answer: Add authenticated approvers, digital signatures, policy-as-code, pull-request integration, central storage, notification hooks, concurrency controls, and stronger parsing of structured reports instead of searching for text markers.

6. FastAPI application and API design

6.1 Q: What is FastAPI?

Answer: FastAPI is a Python framework for building web APIs. It uses Python type hints and Pydantic models to validate requests, generate OpenAPI specifications, and provide an interactive Swagger UI at /docs.

6.2 Q: Why did you choose FastAPI?

Answer: It gives request validation, response schemas, error handling, automatic documentation, and test support with little boilerplate. That keeps the demonstration focused on the workflow rather than framework plumbing.

6.3 Q: What is Swagger UI at /docs?

Answer: It is an interactive browser page generated from the FastAPI API contract. It lists each endpoint, its request body, status codes, response model, and lets a presenter execute real requests without Postman.

6.4 Q: What are the six endpoints?

Answer: POST /todos creates a todo; GET /todos lists them; GET /todos/{id} retrieves one; PUT /todos/{id} updates it; DELETE /todos/{id} removes it; PATCH /todos/{id}/toggle flips completion.

6.5 Q: Why do you use POST for creation?

Answer: POST is the normal HTTP method for creating a new resource when the server generates the resource ID. This route returns 201 Created and the new todo.

6.6 Q: Why does GET /todos return an array?

Answer: The collection endpoint represents all stored todo resources, so it returns a JSON list. An empty database correctly returns an empty list rather than an error.

6.7 Q: What does {id} mean in a route?

Answer: It is a path parameter. The client replaces it with a real integer, such as DELETE /todos/1. FastAPI converts and validates it as an integer before the route code runs.

6.8 Q: Why is PUT used for updates?

Answer: The project specification defines PUT as the endpoint for updating title, done, or both fields. The route supports partial supplied fields and treats an empty body as a no-op by explicit design.

6.9 Q: Why is PATCH used for toggle?

Answer: PATCH is suitable for a focused partial state change. The toggle endpoint changes only the completion status, preserving title and ID.

6.10 Q: Why does DELETE return 204?

Answer: 204 No Content is the standard successful response when a resource was deleted and there is no response body to return. A deliberate test bug changed it to 200, and the test/debug flow caught the contract violation.

6.11 Q: What is response_model=TodoOut?

Answer: It tells FastAPI the JSON shape that must be returned. It documents the endpoint and validates that callers see id as an integer, title as text, and done as a Boolean.

6.12 Q: What errors can a caller receive?

Answer: A missing record produces 404 with detail 'todo not found'. Invalid JSON or invalid field types and values produce FastAPI/Pydantic 422 validation responses. Valid requests return the status code defined for their route.

7. Data validation and database code

7.1 Q: What is Pydantic doing here?

Answer: Pydantic converts incoming JSON into typed Python objects and validates the declared rules before route logic runs. It prevents invalid values from reaching the database.

7.2 Q: How is title validated?

Answer: The title field has a maximum length of 200 characters. A field validator also calls `strip()`, so whitespace-only titles are rejected as empty.

7.3 Q: Why use StrictBool?

Answer: StrictBool requires an actual JSON Boolean true or false. Without it, a permissive conversion might treat strings such as 'yes' as true, which would make the API contract less predictable.

7.4 Q: What happens if title is missing?

Answer: Pydantic cannot construct `TodoCreate` because title is required, so FastAPI returns HTTP 422. No insert query is executed.

7.5 Q: Why use SQLite?

Answer: SQLite is built into Python, requires no database server, stores data in one local file, and is easy for judges to run. It is appropriate for a small reproducible demonstration.

7.6 Q: Why not PostgreSQL or MySQL?

Answer: They are better choices for many production multi-user systems, but they add service setup, credentials, and deployment complexity. The project scope deliberately chose SQLite to focus on the AI workflow.

7.7 Q: What does CREATE TABLE IF NOT EXISTS do?

Answer: It creates the todos table on first startup and safely does nothing on later startups if the table already exists. This allows the app to initialize its local database automatically.

7.8 Q: Why is done stored as 0 or 1?

Answer: SQLite does not have a distinct native Boolean storage class. The database stores 0 or 1, while the Pydantic response model presents the value to API clients as false or true.

7.9 Q: What is conn.commit()?

Answer: Database changes are held in the current transaction until commit is called. commit makes inserts, updates, and deletes durable in the SQLite file.

7.10 Q: Why close connections in finally?

Answer: finally runs whether a route succeeds or raises an error. Closing there prevents file handles or database connections from remaining open after a failed operation.

7.11 Q: What is sqlite3.Row?

Answer: It lets code access a selected row by column name, such as `row['done']`, rather than by numerical index. This makes the code easier to read and less fragile.

7.12 Q: Is the toggle operation safe under high concurrency?

Answer: For this small local demo it is acceptable, but it is a read-then-write sequence. In a production concurrent system, use transactions, an atomic update, or stronger concurrency controls to avoid two simultaneous toggles racing.

8. Security, review, and code quality

8.1 Q: How do you prevent SQL injection?

Answer: All user-supplied values are passed to SQLite using ? placeholders and a separate tuple of values. This means the database treats the value as data, not as part of the SQL instruction.

8.2 Q: Why can you not parameterize a SQL column name in the same way?

Answer: Parameter placeholders are for values, not SQL syntax or identifiers. Therefore dynamically interpolating column names can be risky if they ever become user-controlled.

8.3 Q: What SQL issue did Review Agent find?

Answer: The original PUT route assembled a dynamic SET clause using an f-string for column names. The current field names were hard-coded so it was not directly exploitable, but the pattern could become an injection risk after a future change.

8.4 Q: How was that SQL issue fixed?

Answer: The route now selects one of three fully static UPDATE statements: title only, done only, or both. The values in each statement remain parameterized with question-mark placeholders.

8.5 Q: What is scope drift?

Answer: Scope drift occurs when implementation and the approved requirements no longer match. Review Agent noticed the toggle endpoint and validation changes existed before the locked scope document had been updated, so the scope was formally amended to v2.

8.6 Q: What validation issue was found?

Answer: Create rejected whitespace-only titles, but update originally had a gap. Review found the inconsistency, and TodoUpdate now has a matching validator so both routes enforce the same business rule.

8.7 Q: Are there secrets in the repository?

Answer: The demonstrated application uses a local SQLite file and no cloud credentials or external service keys. A production project would require secret management and scanning policies.

8.8 Q: Why is there no authentication?

Answer: Authentication is intentionally out of scope because the API is a local single-user workflow demonstration. It is not presented as a deployed multi-user product.

8.9 Q: What security additions are needed for production?

Answer: Authentication, authorization, tenant isolation, HTTPS, rate limiting, structured logging, dependency scanning, secret management, audit protection, and a production database are the main next steps.

8.10 Q: Does code review replace automated tests?

Answer: No. Review and tests catch different problems. Review finds design, security, and scope concerns; tests execute behavior and validate responses. The workflow intentionally uses both.

8.11 Q: Why record non-blocking findings?

Answer: Documenting non-blocking findings prevents them from disappearing and helps future maintainers understand trade-offs. It also shows the review role can distinguish a risk worth noting from a release blocker.

8.12 Q: What is a secure answer if a judge asks whether the project is fully secure?

Answer: Say no local demo is fully production-secure. Explain the controls that exist - parameterized values, scoped agent access, review, tests - then name the production controls that remain out of scope.

9. Testing, debugging, and CI

9.1 Q: How many tests do you have?

Answer: The repository documents 23 API tests in `app/test_main.py` and 3 controller tests in `workflow/test_handoff_controller.py`, for 26 total automated tests when run together.

9.2 Q: Why are the API tests independent?

Answer: Testing Agent owns the tests and cannot change application source. The tests use FastAPI TestClient against the real API routes, not just direct unit calls to helper functions.

9.3 Q: How are tests isolated?

Answer: The pytest fixture uses `tmp_path` to create a fresh SQLite database for each test and monkeypatches `DB_PATH`. Therefore tests do not share state and do not alter the real `app/todos.db` demo file.

9.4 Q: What happy paths are covered?

Answer: Every endpoint has a success path: create, list, get, update, delete, toggle, and a smoke test that confirms an item created by POST appears in GET `/todos`.

9.5 Q: What edge cases are covered?

Answer: Tests cover empty list, missing and whitespace titles, a 200-character boundary, overlong titles, numeric title input, non-integer IDs, non-Boolean done values, missing records, no-op updates, double deletion, and both toggle directions.

9.6 Q: What did the deliberate failure prove?

Answer: It proved the failure path works. When DELETE returned 200 instead of 204, the test caught it and Debug Agent traced it to both the decorator and returned response, not merely the visible assertion.

9.7 Q: What is APP_BUG versus TEST_BUG versus SPEC_MISMATCH?

Answer: APP_BUG means implementation violates the intended behavior. TEST_BUG means the test itself is wrong. SPEC_MISMATCH means the requirement and implementation disagree, so the team needs a decision before changing either one.

9.8 Q: Why is a Debug Agent useful if pytest already prints an error?

Answer: Pytest reports the symptom. Debug Agent must inspect the test, specification, and source to identify the underlying cause, category, confidence, and smallest correct repair location.

9.9 Q: What does GitHub Actions do?

Answer: The workflow installs dependencies and runs pytest on pushes and pull requests. It verifies the repository code in CI, but it does not run IBM Bob modes or access the local Bob session.

9.10 Q: Why might a local pytest command need PYTEST_DISABLE_PLUGIN_AUTOLOAD?

Answer: This Windows environment has an unrelated globally installed xonsh pytest plugin that expects an interactive console. Disabling third-party auto-loading avoids that environment problem; it does not change the project tests. A clean virtual environment normally avoids it.

9.11 Q: Does 100 percent passing tests prove the system has no bugs?

Answer: No. Tests provide evidence only for covered behavior. They increase confidence, especially when independent and edge-case focused, but production systems also require security review, load testing, integration testing, and operational monitoring.

9.12 Q: What tests would you add next?

Answer: Concurrency tests for toggle and update, database failure handling, property-based validation tests, OpenAPI contract checks, load tests, security tests, and end-to-end CI tests running in a clean environment.

10. Scope, productionization, and scaling

10.1 Q: What is deliberately out of scope?

Answer: Authentication, multi-user support, pagination, search, a frontend, external databases, Docker, deployment configuration, rate limiting, logging middleware, and observability are explicitly excluded from the demo scope.

10.2 Q: Why exclude so many features?

Answer: A hackathon proof of concept needs a clear boundary. Adding product features would make the demo harder to verify and distract from the primary research question: whether scoped multi-agent collaboration improves the development process.

10.3 Q: Is the API production-ready?

Answer: No. It is a local, single-file SQLite API used to demonstrate the workflow. The workflow principles are useful in production, but the application needs additional product and operational capabilities before deployment.

10.4 Q: How would you productionize the API?

Answer: Move to PostgreSQL, add migrations, authentication and authorization, multi-tenancy if needed, structured logs, metrics, tracing, rate limits, configuration management, backups, deployment, and security monitoring.

10.5 Q: How would you productionize the workflow?

Answer: Use authenticated users, protected Git branches, pull-request gates, signed approvals, immutable audit logs, centralized policy storage, secrets handling, notifications, and integrations with issue trackers and CI systems.

10.6 Q: How would you scale the database?

Answer: Replace local SQLite with a managed relational database such as PostgreSQL, use connection pooling, add indexes based on real access patterns, and introduce pagination/filtering only when the product requirements call for them.

10.7 Q: How would you scale agent orchestration?

Answer: Represent agent tasks and evidence checks as durable workflow events, add queues and retries, track run IDs, enforce policies centrally, and allow parallel tasks only where dependencies are independent.

10.8 Q: Would you use a microservices architecture now?

Answer: No. The application is intentionally small, and a single FastAPI service is the simpler design. Microservices add operational complexity and should be introduced only when domain boundaries and scale justify them.

10.9 Q: What metrics would prove the workflow works at scale?

Answer: Measure defect escape rate, time to diagnose, review findings per change, test coverage, rework, approval latency, cost per change, agent failure rate, and comparison against a one-agent baseline.

10.10 Q: What is the long-term product vision?

Answer: A reusable AI-development governance platform: teams choose a workflow template, create scoped agents, attach evidence gates to pull requests, and obtain auditable records for every AI-assisted change.

10.11 Q: How would this help regulated industries?

Answer: The separation of duties, approval evidence, policy gates, and traceable artifacts align with the kinds of controls required in finance, healthcare, government, and other regulated environments. Domain-specific compliance checks would still need to be added.

10.12 Q: What is the best honest statement about future scope?

Answer: The current project proves the process. The next version should integrate that process with enterprise identity, source-control policies, immutable audit records, security scanning, and deployment pipelines.

11. Comparison, ethics, and credibility

11.1 Q: Is your project better than every alternative?

Answer: We do not claim it is universally better. It is stronger than a single unrestricted AI pass when independence, permissions, and traceability matter. For a trivial task, a single agent may be simpler and faster.

11.2 Q: How is this different from CI/CD?

Answer: CI/CD validates code changes after they exist. Our workflow controls the AI-assisted development process before and around those changes: role boundaries, review artifacts, evidence gates, and human approvals. CI is one independent verification layer inside the overall approach.

11.3 Q: How is this different from a normal human code review?

Answer: The pattern resembles human software engineering controls, but IBM Bob custom modes make AI roles repeatable and can enforce tool/file scopes. Humans remain essential for approvals and for judging whether policy is appropriate.

11.4 Q: Are you replacing developers?

Answer: No. The project is designed to make AI-assisted development more controllable. Humans define scope, approve transitions, inspect evidence, decide trade-offs, and remain responsible for release decisions.

11.5 Q: What happens if an AI report is wrong?

Answer: Reports are evidence, not unquestionable truth. A human can inspect the source, tests, and report. Independent roles reduce the risk of one incorrect claim becoming the only basis for a decision.

11.6 Q: What did you learn from the false Orchestrator-mode information?

Answer: We learned to verify AI-provided claims against actual product behavior, even when a response sounds detailed and confident. We documented the correction and used the supported manual-mode workflow rather than overstating automation.

11.7 Q: Why is honesty about limitations important?

Answer: Overclaiming makes an engineering demo less credible. Being explicit about what Bob controls, what the controller controls, and what remains manual shows that the team understands the real boundary of the solution.

11.8 Q: Could a malicious human bypass the workflow?

Answer: A local repository owner can edit local files, so this is not a full enterprise security boundary. Production mitigation would use repository permissions, branch protections, signed records, identity controls, and external immutable audit storage.

11.9 Q: Could an agent still produce poor code inside its allowed folder?

Answer: Yes. Permissions limit blast radius; they do not guarantee correctness. That is why the workflow also includes review, independent tests, debugging, and human approval.

11.10 Q: What is the ethical benefit of this design?

Answer: It favors transparency and accountability over invisible autonomous behavior. Every important stage produces a human-readable artifact, and a human retains approval authority.

11.11 Q: What is the responsible way to describe AI contribution?

Answer: Say IBM Bob custom modes performed specialized tasks under team-configured roles and permissions. Do not imply the agents are autonomous employees or that CI runs Bob agents unless that is actually true.

11.12 Q: What makes this credible rather than a scripted demo?

Answer: The repository contains the configuration, implementation, tests, review report, debug report, CI workflow, controller code, and audit log. The reported findings correspond to actual code patterns and test behavior.

12. Demo execution, traps, and advanced follow-ups

12.1 Q: What should you show first in a live demo?

Answer: Show the running Swagger page first. Creating and toggling one todo establishes immediately that the product is real. Then explain how IBM Bob and the workflow produced and verified it.

12.2 Q: What exact API sequence is safest?

Answer: POST /todos with a short title, copy the returned ID, GET /todos or PATCH /todos/{id}/toggle, then optionally DELETE it. This is short, visual, and proves creation, persistence, and mutation.

12.3 Q: Should you break code live in a six-minute demo?

Answer: Usually no. It adds risk and consumes time. Show the existing debug_report.md as evidence of the intentional failure-and-recovery run, and run the passing suite live if time permits.

12.4 Q: What should you say if Swagger shows 127.0.0.1?

Answer: 127.0.0.1 means the application runs on this laptop only. It is appropriate for a local demonstration and is not a public deployment address.

12.5 Q: What should you say if asked why tests show 26 but slide says 23?

Answer: There are 23 API tests, which the slide highlights, plus 3 tests for the workflow hand-off controller. Together, the full repository run has 26 tests.

12.6 Q: What if a judge asks to see actual Bob permissions?

Answer: Open .bob/custom_modes.yaml and point to fileRegex under Coding and Testing. Explain that Coding can edit app/ while Testing can write test files and test_report.md but not app/main.py.

12.7 Q: What if a judge asks to see a blocked transition?

Answer: Explain that the controller blocks when evidence is missing or a transition is not in the configured GATES list. Do not alter good project evidence during a short demo unless you have rehearsed the recovery procedure.

12.8 Q: What if a judge challenges the dynamic SQL finding because it was not exploitable?

Answer: Agree with the precise statement. Values were parameterized and current column names were internal, so it was not an active exploit. The review correctly identified a fragile pattern that could become dangerous if a future change made identifiers user-controlled.

12.9 Q: What if a judge asks how you know tests are independent?

Answer: Show the Testing Agent edit scope and the pytest fixture using tmp_path and monkeypatch. The testing role cannot edit source, and each test uses a fresh database rather than a shared application file.

12.10 Q: What if a judge asks for the single biggest limitation?

Answer: The project is a local proof of concept. The workflow demonstrates useful controls, but production use requires identity, stronger audit integrity, deployment, observability, security controls, and a scalable database.

12.11 Q: What if a judge asks for the best future feature?

Answer: Connect the evidence gates to protected pull requests with authenticated approvals and immutable audit records. That turns the proof of concept into a practical enterprise governance workflow.

12.12 Q: How should you answer a question you do not know?

Answer: Do not guess. Say: 'That is not implemented in the current scope. Based on our architecture, I would handle it by ...' Then give a short, reasoned future approach. Honest scope awareness sounds more professional than invented certainty.

12.13 Q: What phrases should you avoid?

Answer: Avoid 'fully autonomous,' 'production-ready,' 'perfectly secure,' and 'Bob automatically switches all agents.' Prefer 'proof of concept,' 'human-approved,' 'scoped permissions,' 'independent evidence,' and 'future hardening.'

12.14 Q: What is the final 20-second close?

Answer: Our contribution is a controlled AI software team, not only generated code. IBM Bob custom modes give specialized roles and permission boundaries; the workflow controller gives evidence gates and human approval; review and testing give independent validation. That is how we make AI-assisted development more trustworthy.

12.15 Q: How do you become confident enough to sound professional?

Answer: Understand the purpose of every component, not every syntax character. Practice explaining one success path, one failure path, one real review finding, and one limitation. Professional answers are precise, evidence-based, and honest about scope.